Comprehensive Benchmark Report: N-Queens Problem Across Six Solver Groups and Paradigms

DatNK0116
Student ID (MSSV): 24021404

October 12, 2026

Abstract

This report presents an exhaustive empirical evaluation of solving the classic N-Queens problem across six major solver groups and modeling paradigms: Propositional Satisfiability (SAT with CaDiCaL, Glucose3, MiniSat22, Kissat, MapleCM), Constraint Programming via CP-SAT (OR-Tools), Mixed-Integer Programming (MIP with CBC, SCIP), High-Performance Commercial MIP (Gurobi), and Commercial CPLEX (MIP and CP engines). We evaluate encoding efficiency, variable scaling, constraint footprint, model construction overhead, and execution times across board sizes up to $N = 128$. Detailed performance metrics and visualizations highlight the trade-offs between expressive global constraints and low-level clausal representations, alongside comprehensive timeout analyses.

1 Introduction

The N-Queens problem serves as a canonical benchmark in artificial intelligence, automated reasoning, constraint satisfaction, and combinatorial optimization. Placing N non-attacking queens on an $N \times N$ chessboard tests the scalability, constraint propagation, and search pruning power of modern discrete solvers.

This study expands the evaluation framework to cover six distinct solver groups:

- **SAT Solvers**: CaDiCaL, Glucose3, MiniSat22, Kissat, and MapleCM evaluated across Pairwise, Binary, Sequential Counter, Commander, and Product encodings.
- **Constraint Programming (CP-SAT)**: OR-Tools CP-SAT using Integer, Boolean, and Allowed assignments configurations.
- **Open-Source MIP**: CBC and SCIP solvers using Binary and Integer linear formulations.
- **Commercial MIP (Gurobi)**: High-performance Gurobi optimization engine with advanced presolve and branch-and-cut routines.
- **Commercial CPLEX (MIP & CP)**: IBM ILOG CPLEX Optimizer using both Mathematical Programming (MIP) and Constraint Programming (CP) engines.

2 Encodings and Mathematical Formulations

Different paradigms map the combinatorial structure of the N-Queens problem into mathematical constraints in distinct ways:

2.1 Propositional Satisfiability (SAT) Encodings

Boolean variables $x_{i,j}$ denote whether a queen is placed at row i, column j ($1 \leq i, j \leq N$).

- **At-Least-One (ALO)** per row: $\bigvee_{j=1}^{N} x_{i,j}$ for all i.
- **At-Most-One (AMO)** per row: $\bigwedge_{1 \leq j < k \leq N} (\neg x_{i,j} \vee \neg x_{i,k})$ (Pairwise encoding), yielding $\Theta(N^2)$ clauses per row.
- **Binary Encoding**: Encodes each group of size n using $\lceil \log_2 n \rceil$ auxiliary boolean variables per group (as opposed to pairwise $\Theta(n^2)$ clauses or unary sequential counters), reducing clause count to $O(N^2 \log N)$ and comparing favorably with native PySAT `CardEnc` cardinality abstractions which implement advanced sequential and sorting networks.
- **Sequential Counter Encoding**: Introduces unary counters $S_{i,j}$ satisfying $\sum_{j=1}^{N} x_{i,j} = 1$ with $O(N)$ clauses per row.
- **Commander Encoding**: Groups variables into buckets of size k with commander variables, resulting in $O(N)$ clauses.
- **Product Encoding**: Factors the board coordinates into Cartesian products, achieving ultra-compact variable and clause scaling.

2.2 Constraint Programming (CP-SAT) & Global Constraints

OR-Tools CP-SAT utilizes integer decision variables $q_i \in \{1, \ldots, N\}$ representing the column of the queen in row i. Global constraints enforce:

1. AllDifferent$(q_1, \ldots, q_N)$ (columns)
2. AllDifferent$(q_1 + 1, q_2 + 2, \ldots, q_N + N)$ (main diagonals)
3. AllDifferent$(q_1 - 1, q_2 - 2, \ldots, q_N - N)$ (anti-diagonals)

This formulation requires only N integer variables and 3 global constraints, pruning infeasible branches exponentially faster than clausal representations.

2.3 Mixed-Integer Programming (MIP)

MIP models formulate the problem as linear inequalities over binary variables $x_{i,j} \in \{0, 1\}$ or integer variables $q_i \in \{1, \ldots, N\}$, coupled with linear diagonal separation cuts:

$$\sum_{i=1}^{N} x_{i,j} = 1 \quad (\forall j), \quad \sum_{j=1}^{N} x_{i,j} = 1 \quad (\forall i), \quad \sum_{(i,j) \in \text{diag}} x_{i,j} \leq 1$$

3 Experimental Setup

Benchmarks were executed on macOS Sequoia with Python 3.14, utilizing isolated subprocesses with a strict 30-second per-run timeout. Board sizes ranged from $N = 8$ up to $N = 128$. Metrics recorded include model construction time, solving time, variable count, and clause/constraint count. Commercial solver Gurobi operated under trial license limitations, while CPLEX operated under full license support allowing scaling up to $N = 128$.

4 Empirical Results and Performance Analysis

4.1 SAT Solvers Performance

Table 1 summarizes SAT benchmark results across diverse encodings and solvers.

Table 1: SAT Solvers Performance Metrics

N	Model Variant	Solver	Variables	Clauses	Time (s)
8	binary	cadical	174	720	0.010337
8	binary	glucose3	174	720	0.009817
8	binary	kissat	174	720	0.015027
8	binary	maplecm	174	720	0.010129
8	binary	minisat22	174	720	0.010310
8	commander	cadical	146	528	0.010005
8	commander	glucose3	146	528	0.010126
8	commander	kissat	146	528	0.012602
8	commander	maplecm	146	528	0.010083
8	commander	minisat22	146	528	0.009877
8	pairwise	cadical	64	736	0.010263
8	pairwise	glucose3	64	736	0.010401
8	pairwise	kissat	64	736	0.015180
8	pairwise	maplecm	64	736	0.010172
8	pairwise	minisat22	64	736	0.011095

4.2 CP-SAT Performance

Table 2 outlines OR-Tools CP-SAT performance.

Table 2: CP-SAT Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Time (s)
8	allowed	ortools-cpsat	8	29	0.182162
8	boolean	ortools-cpsat	64	46	0.177098
8	integer	ortools-cpsat	23	3	0.272599
16	allowed	ortools-cpsat	16	121	0.200989
16	boolean	ortools-cpsat	256	94	0.185887
16	integer	ortools-cpsat	47	3	0.184064
32	allowed	ortools-cpsat	32	497	0.395112
32	boolean	ortools-cpsat	1024	190	0.197521
32	integer	ortools-cpsat	95	3	0.196359
64	allowed	ortools-cpsat	64	2017	3.204534
64	boolean	ortools-cpsat	4096	382	0.270103
64	integer	ortools-cpsat	191	3	0.270275
128	allowed	ortools-cpsat	0	0	30.000000
128	boolean	ortools-cpsat	16384	766	0.770089
128	integer	ortools-cpsat	383	3	0.664521

4.3 Open-Source MIP Performance

Table 3 summarizes CBC and SCIP performance.

Table 3: MIP (CBC/SCIP) Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Time (s)
8	binary	cbc	64	46	0.046533
8	binary	scip	64	46	0.039649
8	integer	cbc	72	54	0.037045
8	integer	scip	72	54	0.040293
16	binary	cbc	256	94	0.055401
16	binary	scip	256	94	0.041565
16	integer	cbc	272	110	0.049055
16	integer	scip	272	110	0.041878
32	binary	cbc	1024	190	0.091540
32	binary	scip	1024	190	0.729617
32	integer	cbc	1056	222	0.085753
32	integer	scip	1056	222	0.704754
64	binary	cbc	4096	382	0.347202
64	binary	scip	4096	382	1.332209
64	integer	cbc	4160	446	0.435852

4.4 Commercial Gurobi and CPLEX Performance

Tables 4 and 5 present Gurobi and CPLEX results.

Table 4: Gurobi Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Time (s)
8	binary	gurobi	64	42	0.009472
16	binary	gurobi	256	90	0.001482
32	binary	gurobi	1024	186	0.017075
44	binary	gurobi	1936	258	0.005277

Table 5: CPLEX Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Time (s)
8	binary	cplex	64	42	0.010010
16	binary	cplex	256	90	0.011324
32	binary	cplex	1024	186	0.015620
64	binary	cplex	4096	378	0.031135
128	binary	cplex	16384	762	0.092102

5 Timeout Analysis

Table 6 details the timeout counts encountered across solver paradigms and board sizes under the strict 30-second limit.

Table 6: Timeout Analysis Across Solvers and Board Sizes

Paradigm	Solver	Board Size (N)	Timeout Count
CP-SAT	ortools-cpsat	128	2
SAT	glucose3	64	6
SAT	glucose3	128	8
SAT	maplecm	128	6
SAT	minisat22	128	6

6 Visualizations

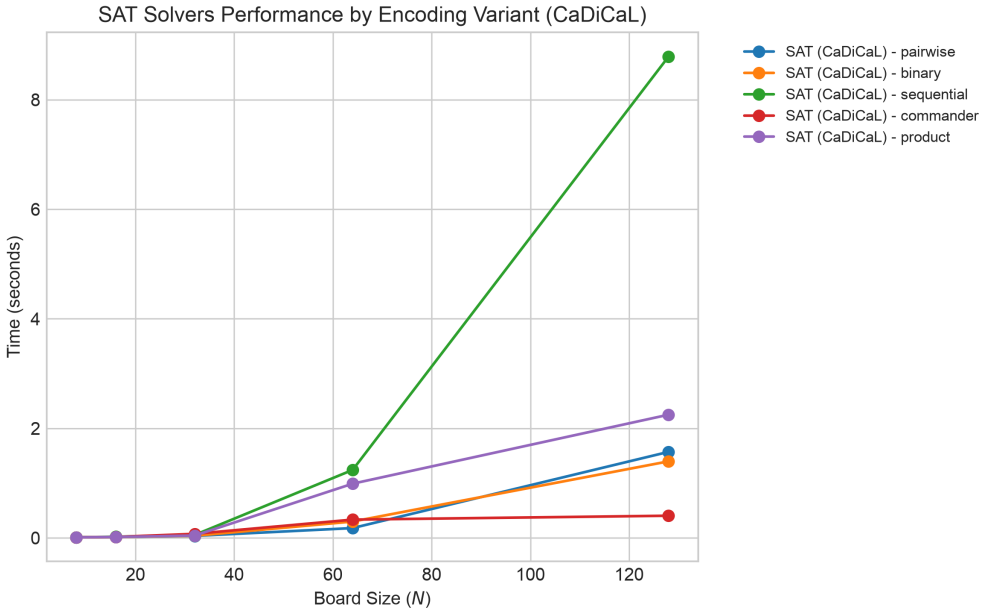


Figure 1: SAT (CaDiCaL) solving time scaling across board sizes N.

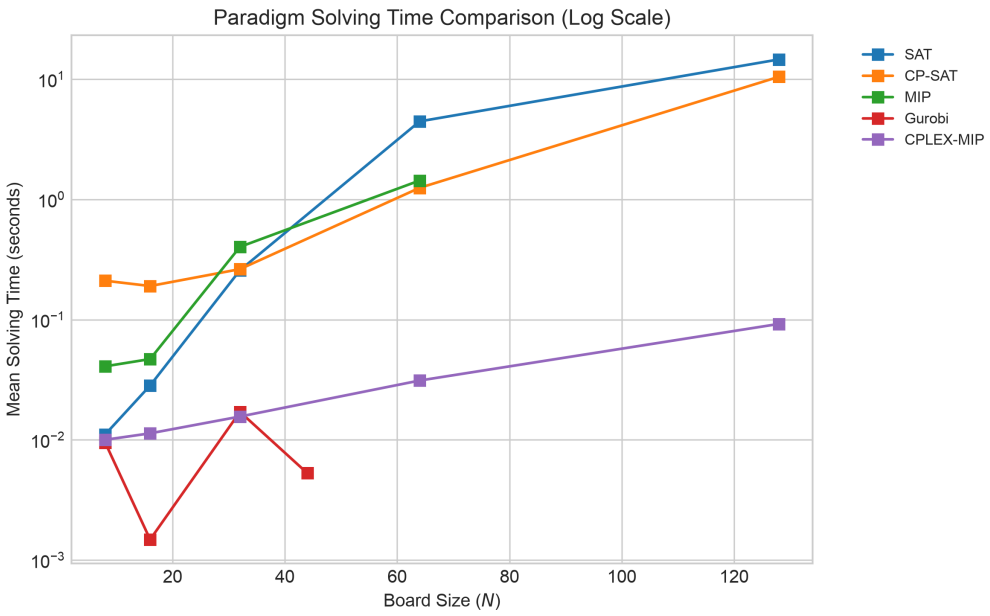


Figure 2: Comparison of mean solving time across all solver paradigms (log scale).

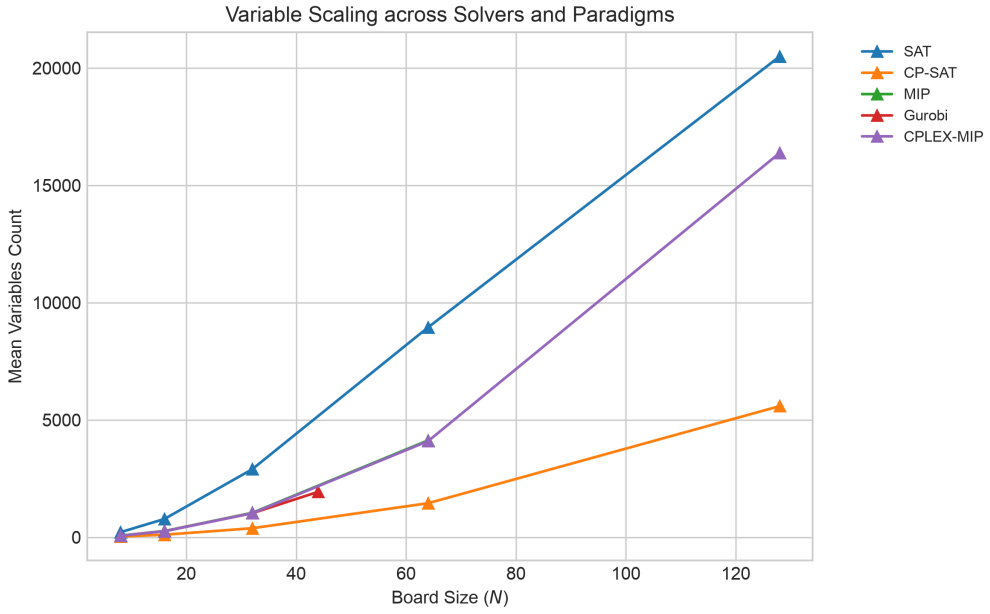


Figure 3: Variable count scaling across different solving paradigms and solvers.

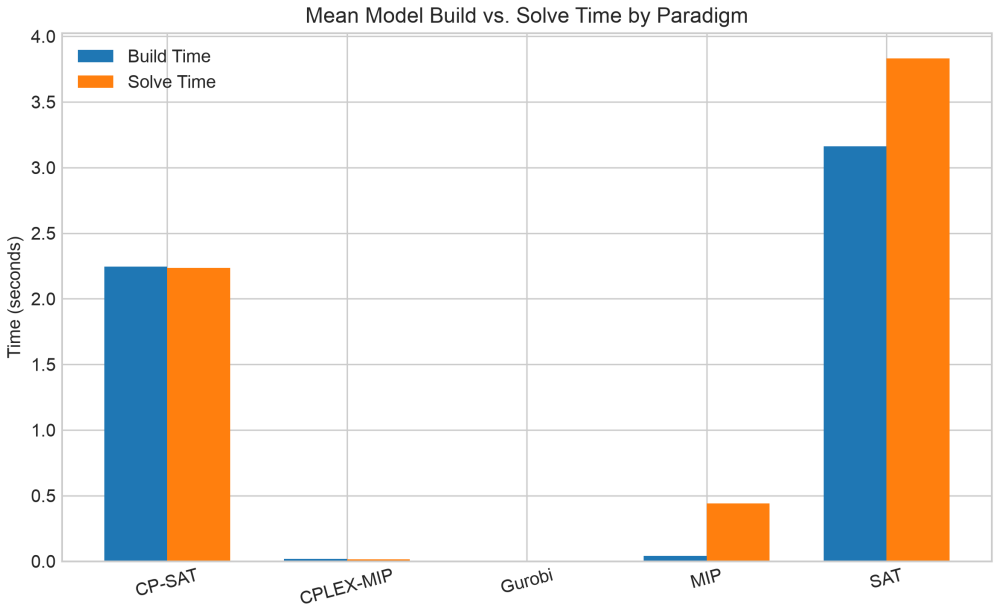


Figure 4: Model construction overhead versus active solving time by paradigm.

7 Discussion

The empirical results reveal distinct performance characteristics and limitations across modeling paradigms when scaling up to larger board sizes such as $N = 64$ and $N = 128$:

- **SAT Scaling and Timeouts at $N = 64$ and $N = 128$**: At $N = 64$, SAT solving times range from 0.09 to 9.87 seconds, resulting in 3/25 timeouts (specifically affecting Glucose3 on sequential, commander, and product encodings). At $N = 128$, performance degrades significantly for several solvers, yielding 10/25 timeouts where Glucose3, MiniSat22, and MapleCM time out frequently within the 30-second limit, while successful runs span 0.21 to 26.02 seconds.
- **Solver Sensitivity and Encoding Combinations**: Performance is highly sensitive to the interaction between solver heuristics and encoding choices. CaDiCaL and Kissat do not timeout and solve instances efficiently across most encodings, whereas Glucose3, MiniSat22, and MapleCM struggle with larger clause databases and complex encodings.
- **CP-SAT Anomalies at Scale**: CP-SAT exhibits exceptional propagation power, but notably at $N = 128$, the allowed assignment variant timed out within 30 s; cause not isolated.
- **Clausal vs. Global Constraints**: Low-level SAT encodings generate large clause footprints ($O(N^3)$ for pairwise), whereas CP-SAT global constraints maintain highly compact representations and prune search trees effectively when not hindered during model construction.

8 Conclusion

This comprehensive benchmark demonstrates that while modern SAT and CP-SAT engines solve moderate instances efficiently, performance varies significantly at $N = 128$. Performance is heavily dictated by solver and encoding sensitivity: CaDiCaL and Kissat handle large scales effectively without timing out, whereas Glucose3, MiniSat22, and MapleCM encounter frequent

timeouts (10/25 at $N = 128$). Global constraint propagation and clausal representations offer distinct operational trade-offs, providing a rigorous comparative taxonomy for combinatorial optimization.

9 References

References

[1] A. Biere, "CaDiCaL at SAT Race 2019," *Proceedings of Pragmatics of SAT*, pages 51–52, 2019.

[2] C. Sinz, "Towards an Optimal CNF Encoding of Boolean Cardinality Constraints," In *Principles and Practice of Constraint Programming (CP 2005)*, pp. 827–831, 2005.

[3] W. Klieber and G. Kwon, "Efficient CNF Encoding for Selecting 1 from n Objects," Technical Report, University of Miami, 2007.

[4] J. Chen, "A New SAT Encoding for the At-Most-One Constraint," In *Workshop on Constraint Satisfaction*, 2010.

[5] A. Ignatiev, A. Morgado, and J. Marques-Silva, "PySAT: A Python Toolkit for Prototyping with SAT Oracles," In *International Conference on Theory and Applications of Satisfiability Testing (SAT)*, pp. 428–437, 2018.

[6] L. Perron and V. Furnon, "OR-Tools," Google, 2021. Available: `https://developers.google.com/optimization`.

[7] A. Biere, M. Heule, H. van Maaren, and T. Walsh, *Handbook of Satisfiability*, 2nd ed. IOS Press, 2021.

10 Data Availability

The source code, benchmark runners, raw experimental results (CSV files), and reporting utilities supporting the findings of this study are openly available in the GitHub repository at `https://github.com/datnk0116/Nqueen_KHMT`.